

GrIOT

Write-up complet

EcowasCTF 2025 • Forensics • 100 pts • 101 solves

Description du challenge

Énoncé

"This data was extracted from ECOWAS IOT gateway device.
I don't really get what happened!"

Le challenge nous donne un fichier binaire extrait d'un appareil IoT de passerelle ECOWAS. L'énoncé laisse entendre que quelque chose d'inhabituel s'est produit dans le firmware — un indice que des données sont cachées ou obfusquées.

Résumé de l'approche

La solution suit 4 étapes clés :

#	Étape	Action
1	Analyse initiale	binwalk sur le fichier — aucun résultat direct
2	Découverte XOR + GZIP	Scan des magic bytes XOR → gzip valide à l'offset 0x1040 avec clé 0xa7
3	Extraction TAR	Décompression → archive TAR contenant un filesystem Linux complet

4

Récupération du flag

Fichier de config avec api_token encodé en Base64

Étape 1 — Analyse initiale du fichier

1

Identification du fichier et binwalk

On commence par les commandes classiques d'analyse de firmware :

```
$ file griot-gw-v1_3.bin
griot-gw-v1_3.bin: data

$ binwalk griot-gw-v1_3.bin
```

```
DECIMAL      HEXADECIMAL  DESCRIPTION
-----
```

```
--
```

```
(aucun résultat)
```

Observation

binwalk ne détecte aucune signature connue dans le fichier brut.
Cela indique que le contenu est probablement chiffré ou obfusqué.

On examine ensuite la structure du fichier manuellement pour comprendre son format. L'analyse des premiers octets révèle un header propriétaire :

```
$ python3 -c "
data = open('griot-gw-v1_3.bin','rb').read()
for i in range(0, 64, 16):
    chunk = data[i:i+16]
    h = ' '.join(f'{b:02x}' for b in chunk)
    a = ' '.join(chr(b) if 32<=b<127 else '.' for b in chunk)
    print(f'{i:04x}: {h:<48} {a}')
```

```
0000: 45 43 46 57 03 00 01 00 47 52 49 4f 54 2d 49 4f  ECFW...GRIOT-IO
0010: 54 2d 37 32 30 00 00 00 c4 f1 67 a7 00 00 00  T-7200.....g....
0020: 40 10 00 00 ad 0b 00 00 4b bb 0e d1 ff ff ff  @.....K.....
```

```
0030: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff .....
```

Offset	Valeur	Description
0x00-0x03	ECFW	Magic bytes — format propriétaire ECOWAS Firmware
0x04-0x07	03 00 01 00	Version du firmware (v3.0.1)
0x08-0x15	GRIOT-IOT-7200	Nom de l'appareil (null-terminé)
0x19-0x1C	c4 f1 67 a7	Champ clé / seed (utilisé pour XOR)
0x20	0x40 (64)	Taille du header
0x2C-0x3F	FF FF FF...	Padding du header

La section payload commence à l'offset 0x40 et contient :

- 0x40-0x7F : JMP court (EB 3E) suivi de 62 NOPs (0x90) — stub bootloader x86
- 0x80-0xE3 : Strings en clair du bootloader
- 0xE4-fin : Données chiffrées (haute entropie)

```
strings griot-gw-v1_3.bin

ECFW
GRIOT-IOT-7200
GRIOT BOOTLOADER v0.4
Initializing hardware...
Loading kernel from flash...
Decompressing rootfs...
```

Étape 2 — Découverte du XOR et du GZIP caché

2

Scan des magic bytes avec déchiffrement XOR

Puisque binwalk échoue sur le fichier brut, on écrit un script pour tester chaque octet (0x00-0xFF) comme clé XOR et chercher la signature gzip (0x1F 0x8B) dans les données déchiffrées :

```
python3 -c "
data = open('griot-gw-v1_3.bin', 'rb').read()
for key in range(256):
```

```
dec = bytes(b ^ key for b in data)
pos = dec.find(b'\x1f\x8b')
if pos != -1 and dec[pos+2] == 8: # méthode deflate
    print(f'gzip VALIDE avec XOR 0x{key:02x} à offset {pos:#x}')
"
```

```
gzip VALIDE avec XOR 0xa7 à offset 0x1040
```

Résultat clé

Clé XOR : 0xa7

Offset du gzip : 0x1040 (4160 en décimal)

Magic bytes trouvés : 1f 8b 08 00 (gzip, méthode deflate)

Étape 3 — Extraction du filesystem

3

Déchiffrement XOR → GZIP → TAR

On déchiffre le fichier avec la clé XOR 0xa7, puis on extrait le contenu gzip :

```
# Déchiffrement XOR et extraction
python3 -c "
data = open('griot-gw-v1_3.bin', 'rb').read()
dec = bytes(b ^ 0xa7 for b in data)
open('decrypted.gz', 'wb').write(dec[0x1040:])
"

# Décompression
gzip -d decrypted.gz

# Listing du TAR
tar tf decrypted
```

```
etc/
etc/config/
etc/init.d/
usr/
usr/bin/
```

```
var/  
var/log/  
opt/  
opt/app/  
etc/passwd  
etc/shadow  
etc/hostname  
etc/config/network.conf  
etc/config/iot_cloud.conf  
etc/config/mqtt.conf  
etc/init.d/S99app  
usr/bin/griot-daemon  
usr/bin/ota-update.sh  
var/log/boot.log  
var/log/syslog  
opt/app/version.txt  
opt/app/.credentials
```

On obtient un filesystem Linux complet issu du gateway IoT. Parmi les fichiers intéressants, plusieurs attirent l'attention :

- `etc/config/iot_cloud.conf` — configuration de connexion cloud
- `opt/app/.credentials` — credentials de développement
- `etc/shadow` — hashes de mots de passe

Étape 4 — Extraction du flag

4 Analyse des fichiers de configuration

On examine le fichier `etc/config/iot_cloud.conf` :

```
tar xOf decrypted etc/config/iot_cloud.conf
```

```
# Griot Cloud Connector - production config  
# DO NOT MODIFY - provisioned at factory  
  
[cloud]  
endpoint = https://api.griot-iot.ecowas.int/v2  
region   = wa-west-1  
protocol = mqttts
```

```
[auth]
method      = token
api_token   = RWNvd2FzQ1RGe2Yxcm13NHlzX3hvc19mc19kdWlwX2dyMTB0fQ==
device_id   = GRT-7200-00AF-B1C3
cert_path    = /etc/ssl/device.pem

[telemetry]
interval_s   = 30
batch_size   = 10
topics       = sensor/temp,sensor/humidity,device/status
```

Le champ `api_token` contient une valeur en Base64. On la décode :

```
echo 'RWNvd2FzQ1RGe2Yxcm13NHlzX3hvc19mc19kdWlwX2dyMTB0fQ==' | base64 -d
```

```
EcowasCTF{f1rmw4r3_xor_fs_dump_gr10t}
```

► FLAG

EcowasCTF{f1rmw4r3_xor_fs_dump_gr10t}

Script de résolution complet

Voici le script Python complet pour reproduire la solution en une seule exécution :

```
import gzip, tarfile, io, base64

# 1. Lecture du fichier
data = open('griot-gw-v1_3.bin', 'rb').read()

# 2. Déchiffrement XOR avec clé 0xa7
decrypted = bytes(b ^ 0xa7 for b in data)

# 3. Extraction du gzip à l'offset 0x1040
gz_data = decrypted[0x1040:]
tar_data = gzip.decompress(gz_data)
```

```
# 4. Lecture du tar en mémoire
tar = tarfile.open(fileobj=io.BytesIO(tar_data))

# 5. Extraction du fichier de config
config = tar.extractfile('etc/config/iot_cloud.conf').read()

# 6. Recherche et décodage du token Base64
for line in config.decode().splitlines():
    if 'api_token' in line:
        token = line.split('=')[1].strip()
        flag = base64.b64decode(token).decode()
        print(f'FLAG: {flag}')
```

```
FLAG: EcowasCTF{f1rmw4r3_xor_fs_dump_gr10t}
```

Leçons et points clés

Ce challenge illustre plusieurs problèmes de sécurité réels dans les firmwares IoT :

Vulnérabilité	Impact
Chiffrement XOR simple (0xa7)	Trivial à casser par brute-force sur 256 valeurs
Clé XOR unique et statique	Aucune rotation de clé, s'applique à tout le firmware
api_token en Base64 dans config	Base64 n'est PAS du chiffrement — données en clair
Credentials en clair dans le FS	etc/shadow, .credentials accessibles après extraction
Filesystem embarqué extractible	Toute la structure Linux du device est récupérable

► FLAG

EcowasCTF{f1rmw4r3_xor_fs_dump_gr10t}

EcowasCTF 2025 — Write-up rédigé avec les étapes de résolution documentées